

# Leaner 在线学习平台产品功能与页面架构设计书

## 1. 概述

本文档旨在规范 Leaner (Lean 语言在线交互式学习平台) 的总体产品架构、页面层级、角色权限规则及核心业务交互流程，为前后端开发提供标准参考协议。

## 2. 站点信息架构 (Sitemap)

平台前端页面层级进行严格的模块化梳理，整体架构树如下：

- 全局框架 (App Shell)
  - 顶部导航栏 (Header)：包含全局面包屑导航 (Breadcrumbs)、系统消息通知中心 (WebSocket 会话红点提示)、用户快捷设置菜单。
  - 侧边导航栏 (Sidebar)：基于用户角色动态渲染的功能入口。
- 一级业务模块
  - 🏠 工作台 (Dashboard)
    - 首页概览：近期待办作业、学习数据统计图表、最新社区动态。
  - 📚 课程与作业 (Course & Assignments)
    - 课程详情页：展示教学大纲、作业列表及 Deadline。
    - 作业答题页 (核心验证模块)：分屏布局布置。包含：Markdown 题目要求区、Monaco Web 代码编辑器、Proof State / 报错信息输出控制台。
    - 智能辅助面板工具箱：内置 LeanSearch 弹窗、In-State 提示组件。
  - 💬 问答社区 (Community)
    - 主题列表页：按课程或作业分类的讨论帖、优秀满分代码展示专区。
    - 帖子详情页：支持富文本解析、代码块高亮及评论盖楼功能。
  - 📁 资源库 (Resources)
    - 学习资源汇聚地：包含环境配置指引 (Windows/Mac/Linux)、官方 Docs 链接、Zulip 论坛跳转直通车、自然数小游戏 (Natural Number Game) 等外链。
  - ⚙️ 教学管理后台 (Teacher/TA Only)
    - 批改工作台：多维度作业筛查、评语录入、打分结算系统。

- 资源配置系统与人员权限管理。

### 3. 基于角色的权限控制矩阵 (RBAC)

系统严格隔离视图与接口，杜绝组件的越权显示。

| 功能模块  |  学生 (Student) |  助教 (Teaching Assistant) |  教师/管理员 (Admin) |
|-------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 作业操作  | 提交作业、<br>保存草稿、<br>自测验证                                                                         |  隐藏<br>(不可看见自己交作业入口)     |  隐藏             |
| 作业批改  | 查看成绩与评语                                                                                        | 评分、填写评语、<br>查看所有学生代码                                                                                      | 同助教，具备终审权                                                                                          |
| 课程发布  | 仅查看选课                                                                                          | 仅查看                                                                                                       | 创建课程、<br>发布带验证脚本的作业                                                                                |
| 社区讨论  | 发帖、互动、<br>代码公开共享                                                                               | 打标签、置顶、删除违规贴、<br>解答                                                                                       | 论坛全量管理                                                                                             |
| 资源库   | 仅浏览与查阅                                                                                         | 上传课件文档、<br>更新外部友情链接                                                                                       | 同助教                                                                                                |
| 人员与系统 |  无权限        |  无权限                   | 赋权（设置某人为助教/<br>封禁用户）                                                                               |

### 4. 核心业务流程规约

#### 4.1 沉浸式作业答题与自测提交流

该流程旨在解决由“纯本地环境配置难”和“盲写提交挫败感强”带来的痛点：

1. **加载阶段**：进入作业答题页，拉取云端的题目描述 (Markdown) 与初始模板骨架代码。
2. **编码与暂存**：用户在 Web 编辑器中编码，系统支持防丢策略的本地缓存/草稿箱存盘。
3. **在线沙盒验证 (Pre-validation)**：
  - 提供独立“运行自测”按钮，前端将代码输送至隔离的 Verifier 异步队列。

- 不记录自测错误次数，实时在辅助面板回显 Error 或 Proof State，辅助学生自行调试。

#### 4. 最终提交 (Submission):

- 代码调通或学生确认后，触发“提交评阅”。此记录将被冻结并推送至助教的批改工作台，同时点亮助教端的提醒红点。

## 4.2 智能外脑辅助流 (Smart Assistance)

在不脱离作业视图的前提下，集成以下定理检索能力：

1. **自然语言检索 (LeanSearch)**: 在右侧面板提供输入框，支持用中文/自然语言搜索并解析返回匹配的 Lean Mathlib 定理名称与签名。
2. **上下文战术推荐 (In-State Search)**: 前端监控编辑器光标或最新 Proof State，后台分析当前证明残缺目标 (Goal)，以非侵入式的 UI (侧方卡片或悬浮器) 提供下一步可能用到的 Tactic。

## 4.3 编译错误智能解析与重构 (Error Handling Optimization)

**核心痛点**: Lean 原生错误信息较为晦涩 (如 type mismatch、failed to synthesize instance)，新手绝大部分时间都在理解错误本身而非解决证明逻辑。如果仅做普通的日志回显，体验依然脱节。

**改进方案**:

1. **常见错误中文转译库**: 建立系统级的错误映射字典。捕获 Top 100 的 Lean 高频编译错误后，自动翻译成通俗易懂的中文，并挂载“可能原因”和“3个最常见的解决方法”。
2. **错误一键寻诊 (One-click Search)**: 在报错控制台的每条 Error 旁提供“一键搜索”按钮，自动提取关键错误结构，跳转至平台内部问答社区或联动 Lean 官方 Zulip 论坛进行相似报错的查询。
3. **重点错误教师批注**: 允许老师或助将某些“历届高发错误”在后台提取并标记为「课程重点」。一旦学生触发该特定报错，立刻弹出提前预设的避坑指南弹窗。

## 4.4 形式化证明逻辑查重系统 (Proof Plagiarism Detection)

**核心痛点**: 普通代码查重工具对于形式化证明 (Lean代码) 完全无效。学生可以通过重命名变量名称、调换证明步骤顺序或替换别名战术来轻易规避字符串匹配，但其证明的逻辑内核实际上完全一样。

**改进方案**:

1. **基于证明项 (Proof Term) 的结构查重**: 摒弃纯文本匹配。后台系统通过提取代码编译后的内核证明项，计算其抽象语法树 (AST) 结构的拓扑相似度，以此精准打击深层逻辑抄袭。
2. **白名单/参考代码排除**: 教师打分系统支持上传该题目的官方参考答案。算法查重时自动过滤或排除与官方参考答案结构相同的合理思路。
3. **证据高亮分析报告**: 系统将高风险的作业对生成可视化的查重报告单，在老师界面的“批改工作台”单独分列，高亮出结构高度同构的证明片段，交由助教/教师进行最终的人工复核。

## 5. UI/UX 体验与基础前端规范

1. **全局防迷航约束**：进入三级页面（如某次作业的某道题目答题器）时，必须配置显著的“< 返回列表”按钮与面包屑路径映射，严禁用户点击后“进退两难”。
2. **排版失控阻断**：定制化平台内部的 Markdown 渲染引擎。强制缩放标题比例（如设定 h1 最高字号为 1.75rem），缩减空白边距，避免大篇幅题目侵占编辑器宝贵的垂直视野。
3. **异步通信与弱网宽容**：验证 Lean 代码极其耗时，点击“运行”后需转动 Loading 状态或采用非阻塞机制，不锁定主线程 UI；若判定编译时间超长（如 5s 以上）需转为后台任务模式运行。